

Information Theory for Multiomics — IsoDecipher Extension Framework

Overview

Information theory provides principled, model-free ways to measure relationships between biological variables. These methods are distribution-agnostic and work naturally with the count-based, high-dimensional data in single-cell multiomics.

1. Shannon Entropy — Cell State Heterogeneity

What it measures: Uncertainty/diversity in a distribution.

Biological application: How diverse is APA usage within a cell population? High entropy = heterogeneous APA usage (mixed cell states) Low entropy = uniform APA usage (committed cell state)

```
import numpy as np
from scipy.stats import entropy

def apa_entropy(adata, gene, cell_type_col='cell_type'):
    """
    Compute Shannon entropy of PUI distribution per cell type.
    High entropy = heterogeneous APA usage
    Low entropy = committed, uniform APA usage
    """
    results = {}
    pui_col = f'{gene}_PUI'

    for ct in adata.obs[cell_type_col].unique():
        mask = adata.obs[cell_type_col] == ct
        pui_vals = adata.obs.loc[mask, pui_col].dropna()

        # Bin PUI into histogram
        hist, _ = np.histogram(pui_vals, bins=20, range=(0, 1), density=True)
        hist = hist + 1e-10 # avoid log(0)

        results[ct] = entropy(hist)
```

```

return pd.Series(results).sort_values(ascending=False)

# Usage
apa_ent = apa_entropy(adata, 'IGHM')
print(apa_ent)

```

2. Jensen-Shannon Divergence (JSD) — APA Divergence Between Cell States

What it measures: Symmetric distance between two distributions. More stable than KL divergence (bounded between 0 and 1).

Biological application:

- Which genes show the most divergent APA between naive vs activated cells?
- Which checkpoint genes (PD-L1, CTLA4) change APA most between tumor states?

```

from scipy.spatial.distance import jensenshannon

def apa_jsd_matrix(adata, gene_list, cell_type_col='cell_type', n_bins=20):
    """
    Compute pairwise JSD of APA distributions between all cell types.
    Returns: gene → (cell_type × cell_type) JSD matrix
    """
    cell_types = adata.obs[cell_type_col].unique()
    results = {}

    for gene in gene_list:
        pui_col = f'{gene}_PUI'
        if pui_col not in adata.obs.columns:
            continue

        # Build PUI distribution per cell type
        distributions = {}
        for ct in cell_types:
            mask = adata.obs[cell_type_col] == ct
            pui_vals = adata.obs.loc[mask, pui_col].dropna()
            hist, _ = np.histogram(pui_vals, bins=n_bins, range=(0, 1), density=True)
            distributions[ct] = hist + 1e-10

        # Pairwise JSD matrix
        n = len(cell_types)

```

```

jsd_matrix = np.zeros((n, n))
for i, ct1 in enumerate(cell_types):
    for j, ct2 in enumerate(cell_types):
        jsd_matrix[i, j] = jensenshannon(
            distributions[ct1], distributions[ct2]
        )

results[gene] = pd.DataFrame(
    jsd_matrix, index=cell_types, columns=cell_types
)

return results

# Find genes with highest APA divergence between cell types
def top_apa_divergent_genes(adata, gene_list, ct1, ct2, n_bins=20):
    """Rank genes by JSD between two cell types."""
    jsd_results = apa_jsd_matrix(adata, gene_list)
    scores = {
        gene: jsd_results[gene].loc[ct1, ct2]
        for gene in jsd_results
    }
    return pd.Series(scores).sort_values(ascending=False)

```

3. Mutual Information (MI) — APA-Expression Coupling

What it measures: How much knowing X reduces uncertainty about Y.

Biological application:

- Which transcription factors are most coupled to APA switching?
- Is CSTF2 expression (APA machinery) mutually informative with PUI changes?
- Find multimodal dependencies: does secretome predict APA state?

```

from sklearn.metrics import mutual_info_score
from sklearn.preprocessing import KBinsDiscretizer

def mutual_info_apa_expression(adata, gene_pui, gene_expr_list, n_bins=10):
    """
    Compute mutual information between PUI of one gene
    and expression of a list of genes.
    Useful for finding TFs or RNA processing factors coupled to APA.
    """
    pui_col = f'{gene_pui}_PUI'

```

```

pui_vals = adata.obs[pui_col].dropna()

# Discretize PUI
est = KBinsDiscretizer(n_bins=n_bins, encode='ordinal', strategy='quantile')
pui_binned = est.fit_transform(pui_vals.values.reshape(-1, 1)).flatten().astype(int)

results = {}
for gene in gene_expr_list:
    if gene not in adata.var_names:
        continue
    expr = adata[pui_vals.index, gene].X.toarray().flatten()
    expr_binned = est.fit_transform(expr.reshape(-1, 1)).flatten().astype(int)

    mi = mutual_info_score(pui_binned, expr_binned)
    results[gene] = mi

return pd.Series(results).sort_values(ascending=False)

# Find which APA machinery genes (CSTF2, NUDT21, etc.) couple to IGHM APA
apa_machinery = ['CSTF2', 'CSTF1', 'NUDT21', 'PABPN1', 'FIP1L1', 'XBP1']
mi_scores = mutual_info_apa_expression(adata, 'IGHM', apa_machinery)
print(mi_scores)

```

4. Transfer Entropy — Causal Direction Along Pseudotime

What it measures: How much does past X reduce uncertainty about future Y? Asymmetric — tells you the DIRECTION of information flow.

Biological application:

- Along pseudotime, does expression of CSTF2 (APA factor) PRECEDE changes in PUI?
- Does transcriptome change come before or after APA switching?
- Causal ordering of multimodal events during differentiation

```

def transfer_entropy_pseudotime(adata, source_gene, target_gene_pui,
                                n_bins=15, lag=2):
    """
    Estimate transfer entropy from source_gene expression to target_gene PUI
    along pseudotime. Positive TE = source causally precedes target.

    lag: pseudotime bins of delay to test
    """
    from itertools import product

```

```

# Sort cells by pseudotime
pt_order = adata.obs['pseudotime'].argsort()

# Get binned source (expression) and target (PUI) series
source = adata[pt_order, source_gene].X.toarray().flatten()
target = adata.obs.iloc[pt_order][f'{target_gene_pui}_PUI'].values

# Discretize
def discretize(x, n):
    bins = np.percentile(x[~np.isnan(x)], np.linspace(0, 100, n+1))
    return np.digitize(x, bins[1:-1])

X = discretize(source, n_bins)
Y = discretize(target, n_bins)

n = len(X) - lag

# P(Y_t+1 | Y_t, X_t) vs P(Y_t+1 | Y_t)
# TE = H(Y_t+1 | Y_t) - H(Y_t+1 | Y_t, X_t)
joint_counts = {}
for i in range(n):
    key = (Y[i], X[i], Y[i+lag])
    joint_counts[key] = joint_counts.get(key, 0) + 1

# Simplified TE estimate using MI
te = mutual_info_score(
    [X[i] for i in range(n)],
    [Y[i+lag] for i in range(n)]
) - mutual_info_score(
    [Y[i] for i in range(n)],
    [Y[i+lag] for i in range(n)]
)

return te

```

5. Total Correlation — Three-Modal Integration

What it measures: Shared information across 3+ variables simultaneously. $TC(X,Y,Z) = H(X) + H(Y) + H(Z) - H(X,Y,Z)$

Biological application:

- Find genes where transcriptome + secretome + APA are ALL simultaneously regulated

- These are the most biologically meaningful genes in SEC-seq + IsoDecipher data

```
def total_correlation_three_modal(adata, gene, n_bins=10):
    """
    Compute total correlation between:
    - Gene expression (transcriptome)
    - Protein secretion (secretome, if SEC-seq available)
    - APA usage (PUI from IsoDecipher)

    High TC = gene is co-regulated across all three modalities
    """
    from scipy.stats import entropy as shannon_entropy

    def discretize(x, n):
        valid = ~np.isnan(x)
        bins = np.percentile(x[valid], np.linspace(0, 100, n+1))
        return np.digitize(x, bins[1:-1])

    # Get three modalities
    expr = adata[:, gene].X.toarray().flatten()
    pui = adata.obs.get(f'{gene}_PUI', pd.Series(np.nan, index=adata.obs_names)).
    secretion = adata.obs.get(f'{gene}_secretion', pd.Series(np.nan, index=adata.

    # Filter valid cells
    valid = ~(np.isnan(expr) | np.isnan(pui) | np.isnan(secretion))
    if valid.sum() < 50:
        return np.nan

    X = discretize(expr[valid], n_bins)
    Y = discretize(pui[valid], n_bins)
    Z = discretize(secretion[valid], n_bins)

    # Marginal entropies
    H_X = shannon_entropy(np.bincount(X) / len(X) + 1e-10)
    H_Y = shannon_entropy(np.bincount(Y) / len(Y) + 1e-10)
    H_Z = shannon_entropy(np.bincount(Z) / len(Z) + 1e-10)

    # Joint entropy H(X,Y,Z)
    joint = pd.Series(list(zip(X, Y, Z))).value_counts(normalize=True)
    H_XYZ = -np.sum(joint * np.log(joint + 1e-10))

    TC = H_X + H_Y + H_Z - H_XYZ
    return TC
```

Summary Table

Method	Measures	IsoDecipher Application
Shannon Entropy	Diversity/uncertainty	Cell state heterogeneity in APA usage
JSD	Distribution distance	APA divergence between cell types
Mutual Information	Statistical dependency	APA-expression coupling, TF discovery
Transfer Entropy	Causal direction in time	Pseudotime ordering of multimodal events
Total Correlation	Multi-variable shared info	Transcriptome + secretome + APA co-regulation

Suggested Implementation Order

1. **JSD** — easiest, most interpretable, direct paper figure
2. **Shannon Entropy** — one function, immediate biological insight
3. **Mutual Information** — find APA machinery coupling (CSTF2, NUDT21)
4. **Total Correlation** — unique to SEC-seq + IsoDecipher combined analysis
5. **Transfer Entropy** — most novel, most publication-worthy

Publication Angle

"IsoDecipher: Information-theoretic dissection of APA dynamics in single-cell multimodal data"

Combining JSD-based APA divergence + MI-based TF coupling + Transfer entropy along pseudotime = a complete, novel analytical framework that no current tool provides.